

Exam 1 Review

Recurrences

Solve $T(n) = 10 T(n/2) + O(n^2)$.

Q1: Solution

$a = 10$, $b = 2$, $d = 2$, and $\log_2 10 \approx 3.32 > 2$.

So $d < \log_b a$: the leaves dominate.

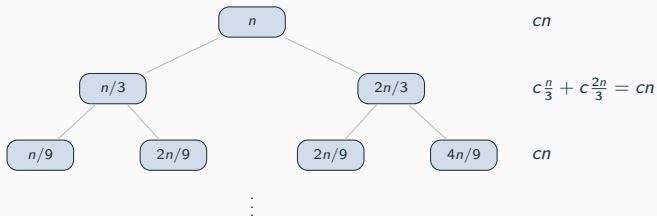
Answer

$$T(n) = O(n^{\log_2 10}) \approx O(n^{3.32})$$

Solve $T(n) = T(n/3) + T(2n/3) + O(n)$.

Q2: Solution

The Master theorem does **not** apply: the two subproblems have different sizes. Draw the recursion tree instead.



Since $\frac{1}{3} + \frac{2}{3} = 1$, every level costs cn , and the depth is $\Theta(\log n)$.

Answer

$$T(n) = \Theta(n \log n)$$

Solve $T(n) = T(n/10) + O(n^2)$.

Q3: Solution

Master theorem with

$$a = 1, \quad b = 10, \quad d = 2, \quad \log_{10} 1 = 0.$$

Since $d = 2 > 0 = \log_b a$, the root dominates.

Answer

$$T(n) = O(n^2)$$

What is the worst-case running time recurrence for this algorithm?

```
Function foo(a[1:n], x)
  If n < 10:
    return a[1] + a[2] + ... + a[n] - x

  If a[n] > 10:
    y = 0
    for i = 1, 2, ..., n/3:
      y = y + a[i]
    return foo(a[1:n/3], y)
  else:
    return a[1] + a[2] + ... + a[n] + x * x
```

Q4: Solution

- base case: $O(1)$
- else branch: $O(n)$, no recursion
- if branch: $O(n)$ for the loop, then **one** call on $n/3$

The recursing branch is the worst case.

Answer

$$T(n) = T(n/3) + O(n) = O(n)$$

True or False

$3^n = O(2^n)$. True or false?

False.

$$\lim_{n \rightarrow \infty} \frac{3^n}{2^n} = \lim_{n \rightarrow \infty} \left(\frac{3}{2}\right)^n = \infty,$$

so no constant c has $3^n \leq c \cdot 2^n$. In fact $3^n = \omega(2^n)$.

$$1^{10} + 2^{10} + 3^{10} + \dots + n^{10} = \Theta(n^{10}).$$

True or false?

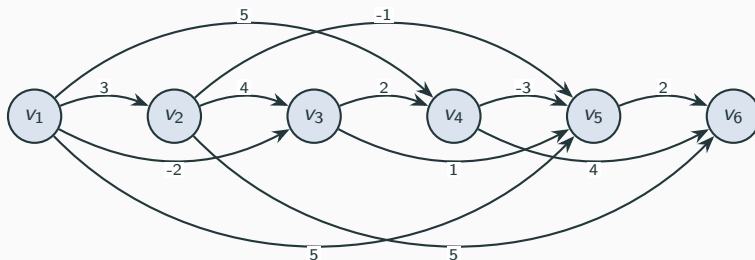
False. It is $\Theta(n^{11})$.

Standard fact: $1^c + 2^c + \dots + n^c = \Theta(n^{c+1})$, here with $c = 10$.

Shortest Paths in a DAG

Q7

The vertices $v_1, \dots, v_6$ are in topological order, so every edge goes left to right. Edge lengths may be **negative**.



Compute the shortest path length from v_1 to each of $v_2, \dots, v_6$.

Q7: Solution

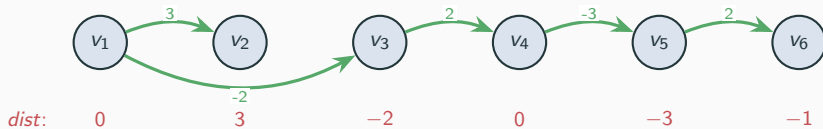
Fill left to right using $dist[v_j] = \min_{v_i \rightarrow v_j} (dist[v_i] + W[v_i, v_j])$.

- $dist[v_2] = 0 + 3 = 3$
- $dist[v_3] = \min(\underbrace{0}_{v_1} - 2, \underbrace{3}_{v_2} + 4) = -2$
- $dist[v_4] = \min(\underbrace{0}_{v_1} + 5, \underbrace{-2}_{v_3} + 2) = 0$
- $dist[v_5] = \min(\underbrace{0}_{v_1} + 5, \underbrace{3}_{v_2} - 1, \underbrace{-2}_{v_3} + 1, \underbrace{0}_{v_4} - 3) = -3$
- $dist[v_6] = \min(\underbrace{3}_{v_2} + 5, \underbrace{0}_{v_4} + 4, \underbrace{-3}_{v_5} + 2) = -1$

Answer

$dist = [0, 3, -2, 0, -3, -1]$ for $v_1, \dots, v_6$

Q7: The Shortest Path Tree



Green edges form the shortest path tree.

Reading DP Tables

Let $K[i, c]$ be the maximum value using items $1, \dots, i$ with capacity c . Suppose

$K[149, 400] = 910$	$K[149, 280] = 700$	$K[148, 400] = 880$	$w[150] = 120$
$K[149, 120] = 310$	$K[149, 250] = 640$	$K[100, 200] = 450$	$v[150] = 260$
$K[148, 280] = 690$	$K[149, 300] = 760$	$w[125] = 75$	$v[125] = 140$

What is $K[150, 400]$?

Q8: Solution

The recurrence looks only at row $i - 1$:

$$K[i, c] = \max \left(\underbrace{K[i - 1, c]}_{\text{skip item } i}, \underbrace{K[i - 1, c - w[i]] + v[i]}_{\text{take item } i} \right).$$

With $i = 150$, $c = 400$, $w[150] = 120$, $v[150] = 260$:

$$K[150, 400] = \max (K[149, 400], K[149, 400 - 120] + 260) = \max (910, 700 + 260).$$

Answer

$$K[150, 400] = \max(910, 960) = 960 \quad (\text{take item 150})$$

Let $ED[i, j]$ be the edit distance between $A[1 \dots i]$ and $B[1 \dots j]$. Suppose $A[7] = T$ and $B[5] = T$, and

$$\begin{array}{llll} ED[6, 5] = 8 & ED[7, 4] = 6 & ED[6, 4] = 5 & ED[5, 5] = 9 \\ ED[7, 3] = 11 & ED[6, 6] = 7 & ED[5, 4] = 4 & ED[7, 6] = 9 \end{array}$$

What is $ED[7, 5]$?

Q9: Solution

$$ED[i, j] = \min \left(\underbrace{ED[i-1, j] + 1}_{\text{delete } A[i]}, \underbrace{ED[i, j-1] + 1}_{\text{insert } B[j]}, \underbrace{ED[i-1, j-1] + \text{diff}(i, j)}_{\text{substitute}} \right).$$

Since $A[7] = B[5] = \text{T}$, we have $\text{diff}(7, 5) = 0$. So

$$ED[7, 5] = \min \left(\underbrace{8+1}_{ED[6,5]}, \underbrace{6+1}_{ED[7,4]}, \underbrace{5+0}_{ED[6,4]} \right).$$

Answer

$$ED[7, 5] = \min(9, 7, 5) = 5$$

Majority Element

Let L, R be a partition of $A[1 : n]$.

If x is the majority of L , then x is always the majority of A .

True or false?

False.

Counterexample. Take $L = [a, a]$ and $R = [b, b, b, b]$.



a is the majority of L ($2 > 1$) but not of A ($2 \not> 3$).

Let L, R be a partition of $A[1 : n]$.

If x is the majority of L *and* the majority of R ,
then x is always the majority of A .

True or false?

True.

Add the two assumptions:

$$\text{count}(x, A) = \text{count}(x, L) + \text{count}(x, R) > \frac{|L|}{2} + \frac{|R|}{2} = \frac{n}{2}. \quad \square$$

Algorithm Design

Q12

A sorted array $A[1 : n]$ of **distinct** elements has been cyclically shifted k positions to the right. Find k in $O(\log n)$ time.

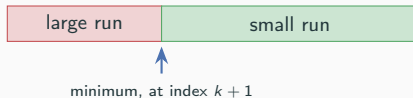


The last k elements have wrapped around to the front.

Q12: Solution

Key observation. The array is two sorted runs, and the **minimum** sits at index $k + 1$. So finding k means finding the minimum's index.

Binary search, comparing $A[mid]$ against the last element $A[hi]$.



Q12: Solution

```
MIN-INDEX(A, lo, hi): // index of the min inside A[lo..hi]
    if lo = hi:
        return lo // base case: one element left

    mid = floor((lo + hi) / 2)
    if A[mid] > A[hi]:
        return MIN-INDEX(A, mid+1, hi) // wrap point is to the right
    else:
        return MIN-INDEX(A, lo, mid) // min is at mid or left of it

FIND-SHIFT(A[1..n]):
    return MIN-INDEX(A, 1, n) - 1
```

Each call discards half the range, so $T(n) = T(n/2) + O(1) = O(\log n)$ by the Master theorem.

The 0-1 knapsack problem with values $v[1 : n]$, weights $w[1 : n]$, and capacity W , plus one extra constraint:

at most k items may be selected.

Design a dynamic programming algorithm that outputs the maximum achievable value.

Q13: Solution

Add a **third dimension** counting how many items we have used.

Definition

$K[i, c, j]$ = maximum value using items $1 \dots i$, capacity c , and at most j items.

The same two choices as before, but taking an item now also spends one of the j slots:

$$K[i, c, j] = \max \left(\underbrace{K[i-1, c, j]}_{\text{skip } i}, \underbrace{K[i-1, c - w[i], j-1] + v[i]}_{\text{take } i, \text{ if } w[i] \leq c, j \geq 1} \right).$$

Base cases $K[0, c, j] = K[i, c, 0] = 0$; the answer is $K[n, W, k]$.

Answer

$O(nWk)$ time

Q13: Pseudocode

```
KNAPSACK-K(v[1..n], w[1..n], W, k):  
    for c = 0 to W:  
        for j = 0 to k:  
            K[0, c, j] = 0  
  
    for i = 1 to n:  
        for c = 0 to W:  
            for j = 0 to k:  
                K[i, c, j] = K[i-1, c, j] // skip item i  
                if j >= 1 and w[i] <= c:  
                    take = K[i-1, c - w[i], j-1] + v[i]  
                    K[i, c, j] = max(K[i, c, j], take)  
  
    return K[n, W, k]
```

Q14: Vasya's Vacation

Vasya has n vacation days. For day i one of four things holds:

a_i	gym	library
0	closed	closed
1	closed	open
2	open	closed
3	open	open

Each day he rests, reads a book (if the library is open), or does sport (if the gym is open).

He will **not read on two consecutive days**, and **not do sport on two consecutive days**. Resting is not an activity, so he may rest any number of days in a row.

Minimise the number of rest days.

Q14: Solution

Tomorrow is constrained only by **what he did today**, so that is the state.

Definition

$dp[i][s]$ = fewest rest days over days $1 \dots i$, where day i ended with activity $s \in \{\text{rest, read, sport}\}$.

$$dp[i][\text{rest}] = 1 + \min(dp[i-1][\cdot])$$

$$dp[i][\text{read}] = \min(dp[i-1][\text{rest}], dp[i-1][\text{sport}]) \quad \text{if } a_i \in \{1, 3\}$$

$$dp[i][\text{sport}] = \min(dp[i-1][\text{rest}], dp[i-1][\text{read}]) \quad \text{if } a_i \in \{2, 3\}$$

Unavailable options are $+\infty$. Note $dp[i][\text{rest}]$ minimises over **all three**, since consecutive rests are allowed. Answer: $\min_s dp[n][s]$, in $O(n)$ time.

Q14: Pseudocode

```
MIN-REST(a[1..n]):  
    dp[rest] = dp[read] = dp[sport] = 0  
  
    for i = 1 to n:  
        new[rest] = 1 + min(dp[rest], dp[read], dp[sport])  
        new[read] = INF  
        new[sport] = INF  
  
        if a[i] = 1 or a[i] = 3: // library open  
            new[read] = min(dp[rest], dp[sport])  
        if a[i] = 2 or a[i] = 3: // gym open  
            new[sport] = min(dp[rest], dp[read])  
  
    dp = new  
  
    return min(dp[rest], dp[read], dp[sport])
```

Q15: One Flip

Given $a_1, \dots, a_n$ with each $a_i \in \{0, 1\}$, choose indices $i \leq j$ and flip every a_k with $i \leq k \leq j$ (that is $x \mapsto 1 - x$).

You must make **exactly one** move. Maximise the number of ones.

a :

1	0	0	1	0	0	1	0	1	0
---	---	---	---	---	---	---	---	---	---

Q15: Solution

Flipping $[i, j]$ trades its ones for its zeros. Set

$$b_k = \begin{cases} +1 & \text{if } a_k = 0, \\ -1 & \text{if } a_k = 1, \end{cases}$$

so the gain from flipping $[i, j]$ is exactly $\sum_{k=i}^j b_k$.

Answer

(total ones) + (maximum **non-empty** subarray sum of b)

Q15: Maximum Subarray Sum

That leaves one subproblem: the largest sum of a non-empty subarray of b .

Definition

$M[k]$ = largest sum of a subarray **ending exactly at** position k .

Such a subarray is either $b[k]$ alone, or the best one ending at $k - 1$ extended by $b[k]$:

$$M[1] = b[1], \quad M[k] = \max(b[k], M[k - 1] + b[k]).$$

The answer is $\max_k M[k]$. Every $M[k]$ contains $b[k]$, so the subarray is non-empty automatically.

One pass, $O(n)$ time and $O(1)$ space if only the running M is kept.

Q15: Maximum Subarray Sum

On the example b :

b :	-1	+1	+1	-1	+1	+1	-1	+1	-1	+1
M :	-1	1	2	1	2	3	2	3	2	3

Each $M[k]$ either restarts at $b[k]$ or extends $M[k - 1]$. For instance $M[3] = \max(1, 1 + 1) = 2$ and $M[4] = \max(-1, 2 - 1) = 1$.

Answer

$\max_k M[k] = 3$, so the answer is $4 + 3 = 7$ ones

Q15: Pseudocode

MAX-ONES(a[1..n]):

total = number of indices i with $a[i] = 1$

for $k = 1$ to n : *// +1 for a zero, -1 for a one*

if $a[k] = 0$: $b[k] = +1$

else: $b[k] = -1$

$M = b[1]$ *// best subarray ending at k*

best = $b[1]$

for $k = 2$ to n :

$M = \max(b[k], M + b[k])$

best = $\max(\text{best}, M)$

return total + best